

UNIT III: Syntax-Directed Translation & Intermediate Code

This unit bridges the parser and the code generator.

Long Answer (10 Marks):

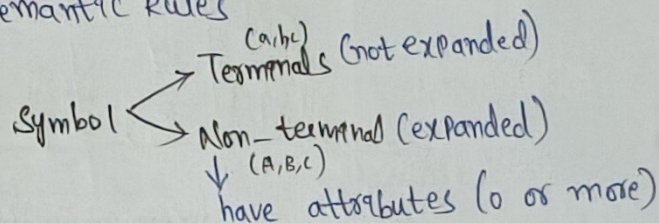
1. **Most Repeated:** Differentiate between **Synthesized Attributes (S-Attributed)** and **Inherited Attributes (L-Attributed)** with examples.
2. Explain **Three-Address Code**. Describe its implementations: **Quadruples, Triples, and Indirect Triples** with an example expression like $a = b * -c + b * -c$.
3. Write the **Syntax Directed Translation (SDT)** scheme for a simple desk calculator or for converting infix to postfix.

* Syntax-Directed Definitions (SDD)

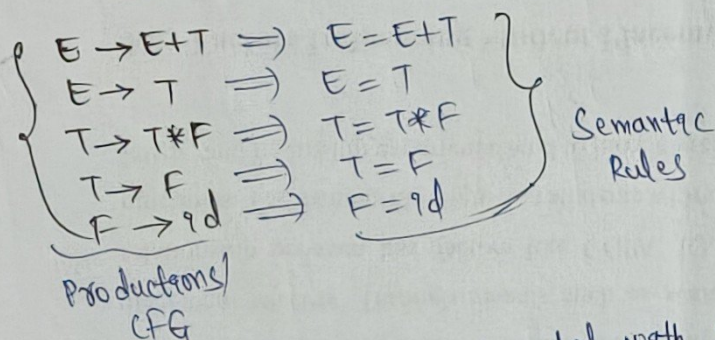
SDD = CFG + Semantic Rules

→ A SDD is a CFG together with Semantic Rules

→ Symbols have attributes



→ Productions have Semantic Rules



⇒ Semantic rules are associated with Productions

⇒ Attributes associated with Symbols

→ Attributes may be numbers, strings, datatypes etc.

Production	Semantic Rule
$E \rightarrow E + T$	$E.val = E.val + T.val$
$E \rightarrow T$	$E.val = T.val$

val - attribute

Types of Attributes :-

① Synthesized Attribute: If a node takes value from its children

- then it is SA

→ Non-terminals on LHS & Production

ex: $A \rightarrow BCD$ | A - Parent node
B, C, D - child nodes

→ uses Bottom-up Parsing

$A.S = B.S$
 $A.S = C.S$
 $A.S = D.S$

} A taking value from B, C, D nodes

②. Inherited Attribute: If a node takes value from its Parent or Siblings 3-2

ex: $A \rightarrow BCD$

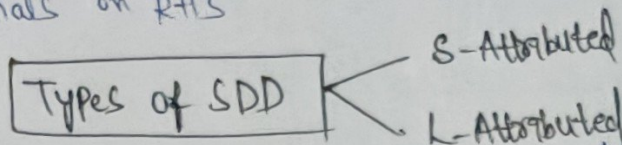
$C.i = A.i$ (C is taking from Parent)

$C.i = B.i$ (C is taking from Sibling nodes)

$C.i = D.i$

→ uses Depth-First Parsing

- Non terminals on RHS



①. S-Attributed SDD / S-Attributed Definitions / S-Attributed grammar

→ A SDD that uses only Synthesized Attributes is called "S-Attributed SDD"

→ It uses Synthesized attributes.

ex: $A \rightarrow BCD$

→ Bottom-up Parsing.

$A.S = B.S$

$A.S = C.S$

$A.S = D.S$

→ Semantic actions are placed at right end of production.

$A \rightarrow BCD \{ A.S = B.S, A.S = C.S, \dots \}$

②. L-Attributed SDD

→ uses both Synthesized & Inherited attributes.

→ But each inherited attribute is restricted

- to inherits from Parent or left Sibling only (not right)

⇒ takes Parent & left Sibling.

ex: $A \rightarrow X \mid Z \{ Y.S = A.S, Y.S = X.S, \cancel{Y.S = Z.S} \}$
} right

→ Semantic actions are placed at anywhere on RHS

$A \rightarrow BC \Rightarrow A \rightarrow \{ \} BC$

$A \rightarrow B \{ \} C$

$BC \{ \}$

③. Three-Address code Representation (TAC)

- Each instruction should contain atmost/maximum 3 address
- Atmost one operator on R.H.S
- Compilers do not directly translate high-level program into machine code
- They first convert the program into an intermediate form.
- Most common intermediate form is Three-Address Code.
- Every instruction contains:
 - one operator
 - maximum 2 operands
 - one result.

Optional

→ 3 main representations:

①. ~~Quadruples~~ Quadruples

②. Triples

③. Indirect Triples

→ Each representations stores the same meaning but in a different format.

①. Quadruples

→ Quadruple has 4 fields: (operator, argument 1, argument 2, result)

ex: $(+, a, b, t1)$

It means $\Rightarrow t1 = a + b$

Structure: | Operator | Arg 1 | Arg 2 | Result |

→ Simple → slow
→ easy → extra space

Quadruple Representation

Step	operator	Arg 1	Arg 2	Result
0	*	b	c	t1
01	+	a	t1	x

ex: $x = a + b * c$

TAC: $t1 = b * c$
 $x = a + t1$

②. Triples : remove the result field.

→ The o/p is referred by "instruction index"

(index, operator, arg 1, arg 2)

ex: $x = a + b * c$

Triple Representation

Index	operator	Arg 1	Arg 2
0	*	b	c
1	+	a	(0)

TAC:

$t1 = b * c$

$t2 = a + t1$

→ Saves memory, one column removed

bcz, in table t1 is in '0' index

3. Indirect Triples

3-3

→ It use ② separate tables

①, Instruction Table : contains triple.

②, pointer Table : contains execution order.

ex: $x = a + b * c$

Instruction Table

Index	operator	arg1	arg2
0	*	b	c
1	+	a	(0)
2	=	(1)	x

Pointer Table

PT Index	Instruction Index
0	0
1	1
2	2

ex: $a = b * -c + b * -c$
Quadruple
 $t1 = -c$
 $t2 = b * t1$
 $t3 = -c$ (or)
 $t4 = b * t3$
 $t5 = t2 + t4$

step	operator	arg 1	arg 2	Result
0	-	c		t1
1	*	b	t1	t2
2	-	c		t3
3	*	b	t3	t4
4	+	t2	t4	t5

Triple

step	operator	arg 1	arg 2
0	-	c	
1	*	b	(0)
2	-	c	
3	*	b	(2)
4	+	(1)	(3)

Indirect Triple

Pointel Table

PT Index	Instruction Table
100	(0)
101	(1)
102	(2)
103	(3)
104	(4)

Instruction Table

Index	opera	arg 1	arg 2
0	-	c	
1	*	b	(0)
2	-	c	
3	*	b	(2)
4	-	(1)	(3)

* Evaluation orders for SDD's (decoding order)

3-3

- we use Dependency graphs
- to determine an evaluation order
- Dependency Graphs are useful for determining Evaluation order for attributes in a parse tree. (which can be first)
- DG represent the flow of information
 - among the attributes in a parse tree
- Annotated Parse tree shows the values of attributes
- Dependency graph determines how those values can be computed.

Ex: Productions

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow \text{digit}$

Semantic Rules

$E.val = E.val + T.val$

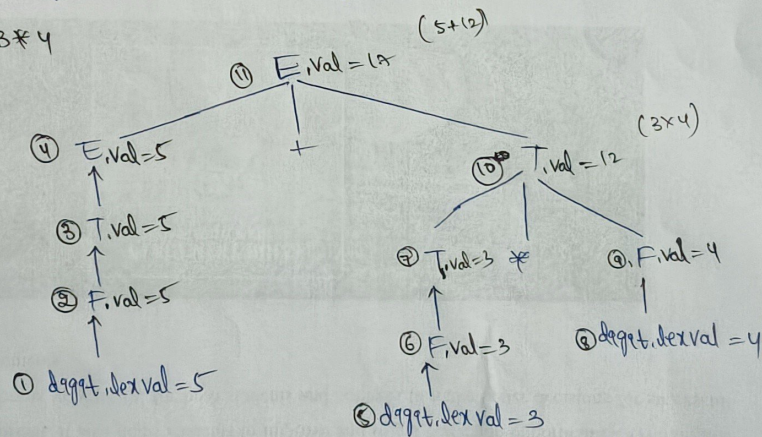
$E.val = T.val$

$T.val = T.val * F.val$

$T.val = F.val$

$F.val = \text{digit.lexvalue}$

Ex: $5 + 3 * 4$



* Applications of SDT

3-4

- Construction of Symbol Tables
- Type checking
- Intermediate Code Generation
- Construction of Parse Trees

* SDT Schemes

- All applications of SDD can be implemented using SDT Schemes
- SDT's are implemented during parsing

Def: A SDT is a CFG where Semantic actions are embedded directly into the production rules.

→ SDT tells:

- how translation happens during parsing
- Specifies where & when each action must be executed.

→ SD is used to generate:

- ~~general~~ - intermediate code
- build syntax trees
- check types

→ An SDT Scheme is a CFG + Semantic actions written in braces { }

→ SDT defines the execution order.